TP FireStorm

Analyse Android, Hooking Frida et authentification Firebase

Amira Ezbiri

RÉSUMÉ

Ce rapport présente une démarche complète d'analyse d'une application Android contenue dans un challenge de sécurité. L'objectif consiste à identifier une fonction de génération de mot de passe, à forcer son exécution avec Frida, puis à utiliser le résultat pour s'authentifier sur Firebase et récupérer une donnée protégée. Le document décrit l'environnement de travail, l'analyse statique de l'APK, l'interception dynamique, l'exploitation des paramètres Firebase et la validation finale du flag.

Filière : Génie Cyber Défense et Systèmes de Télécommunications Embarqués

Table des matières

1 Introduction **3**

2 Objectifs du TP **3**

3 Environnement de travail **3**

4 Mise en place de l'émulateur **3**

5 Analyse statique de l'APK **4**

5.1 Structure générale observée 4

5.2 Compréhension de la méthode Password() 4

5.3 Indice important : méthode non appelée 4

6 Extraction des paramètres Firebase **4**

7 Analyse dynamique avec Frida **4**

7.1 Principe du hooking 4

7.2 Script Frida utilisé (extract_secret.js) 4

7.3 Lancement et Résultat 5

8 Authentification Firebase **5**

9 Discussion et Conclusion **5**

Annexe A - Commandes utiles **6**

Annexe B - Livrables **6**

1 Introduction

Les challenges de reverse Android sont utiles pour comprendre comment une application protège ses données, comment elle organise sa logique métier et comment elle communique avec des services distants. Dans ce TP, l'application FireStorm contient une logique de génération de mot de passe présente dans le code, mais non déclenchée par l'interface classique. Le but n'est pas de casser un chiffrement complexe, mais plutôt de repérer une piste discrète dans le code, puis d'exécuter la bonne méthode au bon moment.

L'approche suivie repose sur deux axes complémentaires :

— l'analyse statique avec **Jadx** pour comprendre la structure de l'application ;

— l'analyse dynamique avec **Frida** pour invoquer une méthode Java au runtime.

2 Objectifs du TP

Les objectifs précis du challenge sont les suivants :

1. Installer l'application sur un environnement Android de test ;
2. Identifier les classes et méthodes pertinentes dans l'APK ;
3. Retrouver la méthode `Password()` ;
4. Forcer son exécution avec Frida ;
5. Récupérer le mot de passe généré dynamiquement ;
6. Utiliser les paramètres Firebase trouvés dans l'application ;
7. S'authentifier sur le service distant ;
8. Récupérer le flag stocké dans la base de données.

3 Environnement de travail

Le TP a été réalisé avec les outils suivants :

Composant	Utilisation
Android Studio	Création et lancement de l'émulateur Android
ADB	Installation de l'APK et interaction avec l'appareil
Jadx-GUI	Décompilation et lecture du code Java
Frida	Hooking Java et appel de méthodes au runtime
Python 3	Automatisation de l'authentification Firebase
pyrebase	Accès simple à Firebase depuis Python

4 Mise en place de l'émulateur

La première étape consiste à ouvrir Android Studio puis à créer un appareil virtuel via le gestionnaire de périphériques. Une fois l'émulateur lancé, il devient possible d'utiliser ADB pour installer l'application :

```
# Verification de la connexion
adb devices -l
# Installation de l'application
adb install FireStorm.apk
```

5 Analyse statique de l'APK

L'analyse statique a été effectuée avec **Jadx**. L'objectif est d'explorer le code décompilé et les ressources internes.

5.1 Structure générale observée

Les éléments les plus importants repérés dans le projet sont :

- Le package principal : `com.pwnsec.firestorm` ;
- La classe principale : `MainActivity` ;
- La méthode cible : `Password()`.

5.2 Compréhension de la méthode Password()

La méthode `Password()` assemble des chaînes statiques lues depuis `strings.xml` avec une partie dynamique obtenue par une fonction native. La logique du secret n'est pas complètement visible dans le Java car une partie est déportée dans une bibliothèque native (`.so`).

5.3 Indice important : méthode non appelée

Un élément essentiel est que `Password()` n'est **pas appelée** par le flux normal de l'application (pas de bouton, pas d'appel dans `onCreate`). Frida est donc indispensable pour invoquer la méthode manuellement en mémoire.

6 Extraction des paramètres Firebase

L'examen de `strings.xml` permet de retrouver les identifiants de connexion : API key, adresse e-mail (`TK757567@pwnsec.xyz`), et l'URL de la *Realtime Database*.

7 Analyse dynamique avec Frida

7.1 Principe du hooking

Nous utilisons `Java.choose()` pour rechercher une instance vivante de `MainActivity` déjà chargée en mémoire et forcer l'exécution de sa méthode membre.

7.2 Script Frida utilisé (extract_secret.js)

```
Java.perform(function () {
    function triggerPasswordBuild() {
        console.log("[*] Recherche d'une instance de MainActivity...");
        Java.choose("com.pwnsec.firestorm.MainActivity", {
```

```
            onMatch: function (activityInstance) {
                console.log("[+] Instance trouvee !");
                try {
                    var recoveredPassword = activityInstance.Password();
                    console.log("[+] Mot de passe recupere : " + recoveredPassword);
                } catch (err) {
                    console.log("[-] Erreur : " + err);
                }
            },
            onComplete: function () { console.log("[*] Fin de recherche."); }
        });
    }
    setTimeout(triggerPasswordBuild, 3500);
});
```

7.3 Lancement et Résultat

```
frida -U -f com.pwnsec.firestorm -l extract_secret.js --no-pause
```

Le terminal affiche alors le mot de passe généré dynamiquement par la bibliothèque native.

8 Authentification Firebase

Avec le mot de passe extrait, nous utilisons un script Python (`pyrebase`) pour récupérer le flag.

```
import pyrebase

config = {
    "apiKey": "AIzaSyAXsK0qsx4RuLSA9C8IPSWd0eQ67HVHuJY",
    "authDomain": "firestorm-9d3db.firebaseapp.com",
    "databaseURL": "https://firestorm-9d3db-default-rtdb.firebaseio.com",
    "projectId": "firestorm-9d3db"
}

firebase = pyrebase.initialize_app(config)
auth = firebase.auth()
user = auth.sign_in_with_email_and_password("TK757567@pwnsec.xyz", "MOT_EXTRAIT")

db = firebase.database()
print("[+] Flag :", db.get(user["idToken"]).val())
```

9 Discussion et Conclusion

Ce TP illustre que la sécurité par l'obscurité (cacher une fonction) est insuffisante sur Android. L'utilisation combinée de **Jadx** pour la reconnaissance et **Frida** pour l'exploitation permet de contourner les protections basées sur le flux d'exécution standard. En conclusion, les secrets critiques ne devraient jamais être reconstruits ou stockés côté client sans protections supplémentaires.

Annexe A - Commandes utiles

```
adb install FireStorm.apk
frida-ps -Uai
frida -U -f com.pwnsec.firestorm -l extract_secret.js
```

Annexe B - Livrables

- `scripts/extract_secret.js` : Script de hooking.
- `scripts/recover_flag.py` : Script d'extraction Firebase.
- `report/rapport.pdf` : Ce document.